

VISVESVARAYA TECHNOLOGICAL UNIVERSITY

“JnanaSangama”, Belgaum -590014, Karnataka.



LAB REPORT

On

OPERATING SYSTEMS (23CS4PCOPS)

Submitted by

ANANYA RANJITH (1BM24CS041)

**in partial fulfillment for the award of the degree of
BACHELOR OF ENGINEERING
in
COMPUTER SCIENCE AND ENGINEERING**

B.M.S. COLLEGE OF ENGINEERING

(Autonomous Institution under VTU)

BENGALURU-560019

Feb-2026 to June-2026

B. M. S. College of Engineering,

Bull Temple Road, Bangalore 560019

(Affiliated To Visvesvaraya Technological University, Belgaum)

Department of Computer Science and Engineering

CERTIFICATE

**B. M. S. College of Engineering,
Bull Temple Road, Bangalore 560019
(Affiliated To Visvesvaraya Technological University, Belgaum)
Department of Computer Science and Engineering**



This is to certify that the Lab work entitled “**OPERATING SYSTEMS**” carried out by ANANYA RANJITH (**IBM24CS041**), who is a bonafide student of **B. M. S. College of Engineering**. It is in partial fulfillment for the award of **Bachelor of Engineering in Computer Science and Engineering** of the Visvesvaraya Technological University, Belgaum during the year 2025-26. The Lab report has been approved as it satisfies the academic requirements in respect of Operating Systems Lab - (**23CS4PCOPS**) work prescribed for the said degree.

Rajeshwari Madli
Assistant Professor
Department of CSE
BMSCE, Bengaluru

Dr. Kavitha Sooda
Professor and Head
Department of CSE
BMSCE, Bengaluru

Index

Sl. No.	Experiment Title	Page No.
1	Write a C program to simulate the following CPU scheduling algorithm to find turnaround time and waiting time. a) FCFS b) SJF c) Priority d) Round Robin .	4 - 13
2	Write a C program to simulate a multi-level queue scheduling algorithm considering the following scenario. All the processes in the system are divided into two categories – system processes and user processes. System processes are to be given higher priority than user processes. Use FCFS scheduling for the processes in each queue.	14 - 17
3	Write a C program to simulate Real-Time CPU Scheduling algorithms: a) Rate- Monotonic b) Earliest-deadline First c) Proportional scheduling	18 - 25
4	Write a C program to simulate: a) Producer-Consumer problem using semaphores. b) Dining-Philosopher's problem	26 - 32
5	Write a C program to simulate: a) Bankers' algorithm for the purpose of deadlock avoidance. b) Deadlock Detection	33 - 39
6	Write a C program to simulate the following contiguous memory allocation techniques. a) Worst-fit b) Best-fit c) First-fit	40 - 48
7	Write a C program to simulate page replacement algorithms. a) FIFO b) LRU c) Optimal .	49 - 55

Course outcomes:

CO1	Analyze time complexity of algorithms using asymptotic notations.
CO2	Apply various algorithm design techniques for the given problem.
CO3	Evaluate the appropriate algorithm/design technique for solving a given problem.
CO4	Conduct practical experiments by applying appropriate algorithm design techniques to solve problems.

Lab program 1:

Write a C program to simulate the following CPU scheduling algorithm to find turnaround time and waiting time.

a) FCFS b) SJF c) Priority d) Round Robin

a) FCFS

```
#include <stdio.h>

int main() {
    int n, i;
    printf("Enter number of processes: ");
    scanf("%d", &n);
    int bt[n], wt[n], tat[n];
    printf("Enter Burst Times:\n");
    for(i = 0; i < n; i++) {
        printf("P%d: ", i + 1);
        scanf("%d", &bt[i]);
    }
    wt[0] = 0;
    for(i = 1; i < n; i++) {
        wt[i] = wt[i - 1] + bt[i - 1];
    }
    for(i = 0; i < n; i++) {
        tat[i] = wt[i] + bt[i];
    }
    float avg_wt = 0, avg_tat = 0;
    printf("\nProcess\tBurst Time\tWaiting Time\tTurnaround Time\n");
    for(i = 0; i < n; i++) {
        printf("P%d\t%d\t%d\t%d\n",
            i + 1, bt[i], wt[i], tat[i]);
        avg_wt += wt[i];
        avg_tat += tat[i];
    }
    avg_wt /= n;
    avg_tat /= n;
    printf("\nAverage Waiting Time = %.2f", avg_wt);
    printf("\nAverage Turnaround Time = %.2f\n", avg_tat);
    return 0;
}
```

Output:

```
Default: C:\Users\anany\Desktop\OS\FCFS.exe
ctrl+alt+t

C:\Users\anany\Desktop\OS\ >
Enter number of processes: 4
Enter Burst Times:
P1: 5
P2: 3
P3: 8
P4: 6

Process Burst Time    Waiting Time    Turnaround Time
P1      5             0              5
P2      3             5              8
P3      8             8             16
P4      6            16             22

Average Waiting Time = 7.25
Average Turnaround Time = 12.75

Process returned 0 (0x0)   execution time : 17.902 s
Press any key to continue.
```

Observation:

OS lab

FCFS

```
#include <stdio.h>
struct process {
    int pid;
    int at;
    int bt;
    int ct;
    int tat;
    int wt;
};

int main() {
    int n, i;
    int c_time = 0;
    int total_wt = 0, total_tat = 0;
    printf("Enter no. of processes: ");
    scanf("%d", &n);
    struct process p[n];
    for(i=0; i<n; i++) {
        printf("Enter AT and BT for process %d: ", i+1);
        scanf("%d %d", &p[i].at, &p[i].bt);
    }
    for(i=0; i<n; i++) {
        for(j=i+1; j<n; j++) {
            if(p[j].at < p[i].at) {
                temp = p[i];
                p[i] = p[j];
                p[j] = temp;
            }
        }
    }
    for(i=0; i<n; i++) {
        if(i==0) {
            p[i].wt = 0;
            p[i].tat = p[i].at;
            p[i].ct = p[i].bt;
            total_wt += p[i].wt;
            total_tat += p[i].tat;
        } else {
            p[i].wt = p[i-1].ct;
            p[i].tat = p[i].at + p[i].wt;
            p[i].ct = p[i].bt;
            total_wt += p[i].wt;
            total_tat += p[i].tat;
        }
        printf("%d\t%d\t%d\t%d\t%d\t%d\n", p[i].pid, p[i].at, p[i].bt, p[i].ct, p[i].tat, p[i].wt);
    }
    printf("\n Average WT = %.2f", (float) total_wt / n);
    printf("\n Average TAT = %.2f", (float) total_tat / n);
    return 0;
}
```

OUTPUT

PID	AT	BT	CT	TAT	WT
1	0	5	5	5	0
2	0	3	8	8	5
3	0	8	16	16	8

Average WT = 4.33
Average TAT = 9.67

PID	AT	BT	CT	TAT	WT
1	6	8	14	8	0
2	6	8	22	16	8
3	7	9	31	24	15

Average WT = 7.67
Average TAT = 16.00

b) SJF

```
#include <stdio.h>

int main() {
    int n, i, j;
    printf("Enter number of processes: ");
    scanf("%d", &n);
    int bt[n], wt[n], tat[n], p[n];
    printf("Enter Burst Times:\n");
    for(i = 0; i < n; i++) {
        p[i] = i + 1;
        printf("P%d: ", i + 1);
        scanf("%d", &bt[i]);
    }
    for(i = 0; i < n - 1; i++) {
        for(j = i + 1; j < n; j++) {
            if(bt[i] > bt[j]) {
                int temp;
                temp = bt[i];
                bt[i] = bt[j];
                bt[j] = temp;
                temp = p[i];
                p[i] = p[j];
                p[j] = temp;
            }
        }
    }
    wt[0] = 0;
    for(i = 1; i < n; i++) {
        wt[i] = wt[i - 1] + bt[i - 1];
    }
    for(i = 0; i < n; i++) {
        tat[i] = wt[i] + bt[i];
    }
    float avg_wt = 0, avg_tat = 0;
    printf("\nProcess\tBurst Time\tWaiting Time\tTurnaround Time\n");
    for(i = 0; i < n; i++) {
        printf("P%d\t%d\t%d\t%d\n",
            p[i], bt[i], wt[i], tat[i]);
        avg_wt += wt[i];
        avg_tat += tat[i];
    }
}
```

```

avg_wt /= n;
avg_tat /= n;
printf("\nAverage Waiting Time = %.2f", avg_wt);
printf("\nAverage Turnaround Time = %.2f\n", avg_tat);
return 0;
}

```

Output:

```

Default: C:\Users\anany\Desktop\OS\SJF.exe
ctrl+alt+j

C:\Users\anany\Desktop\OS\SJF.exe

Enter number of processes: 4
Enter Burst Times:
P1: 6
P2: 8
P3: 7
P4: 3

Process Burst Time    Waiting Time    Turnaround Time
P4      3             0              3
P1      6             3              9
P3      7             9             16
P2      8            16             24

Average Waiting Time = 7.00
Average Turnaround Time = 13.00

Process returned 0 (0x0)   execution time : 18.303 s
Press any key to continue.

```

Observation:

SJF

```

#include <stdio.h>
#define process 4
int pid,
int bt,
int at,
int ct,
int tat,
int wt,
int min_bt,
int remaining_bt,
int completed;

void NonPreemptive (struct Process p[4], int n) {
    int current_time = 0, completed = 0;
    float total_wt = 0, total_tat = 0;
    while (completed < n) {
        int index = -1;
        int min_bt = 9999;
        for (int i = 0; i < n; i++) {
            if (p[i].ct < current_time && p[i].completed == 0) {
                if (p[i].bt < min_bt) {
                    min_bt = p[i].bt;
                    index = i;
                }
            }
        }
        if (index != -1) {
            current_time += p[index].bt;
            p[index].ct = current_time;
            p[index].tat = p[index].ct - p[index].at;
            p[index].wt = p[index].tat - p[index].bt;
            p[index].completed = 1;
            completed++;
        }
    }

```

SURTAGM

```

total_wt += p[index].wt;
total_tat += p[index].tat;
}
else {
    current_time++;
}
}

printf("\n Average Waiting Time = %.2f", total_wt / n);
printf("\n Average TAT = %.2f", total_tat / n);
printf("\n PID | AT | BT | CT | WT | Tat |");
for (int i = 0; i < n; i++) {
    printf(" %d | %d | %d | %d | %d | %d |",
           p[i].pid, p[i].at, p[i].bt,
           p[i].ct, p[i].wt, p[i].tat);
}
}
}

OUTPUT:
Enter number of processes: 4
Process P1
Arrival Time: 0
Burst Time: 6
Process P2
Arrival Time: 0
Burst Time: 8
Process P3
Arrival Time: 0
Burst Time: 7
Process P4
Arrival Time: 5
Burst Time: 3

```

SURTAGM

PID	AT	BT	CT	WT	TAT
P1	0	3	3	0	3
P2	0	4	12	8	12
P3	0	7	16	9	16
P4	5	3	16	11	19

Average Waiting Time: 4.00 ms
Average TAT: 13.00 ms

c) Priority

```
#include <stdio.h>

int main() {
    int n, i, j;
    printf("Enter number of processes: ");
    scanf("%d", &n);
    int bt[n], pr[n], wt[n], tat[n], p[n];
    printf("Enter Burst Time and Priority:\n");
    for(i = 0; i < n; i++) {
        p[i] = i + 1;
        printf("P%d Burst Time: ", i + 1);
        scanf("%d", &bt[i]);
        printf("P%d Priority: ", i + 1);
        scanf("%d", &pr[i]);
    }
    for(i = 0; i < n - 1; i++) {
        for(j = i + 1; j < n; j++) {
            if(pr[i] > pr[j]) {
                int temp;
                temp = pr[i];
                pr[i] = pr[j];
                pr[j] = temp;
                temp = bt[i];
                bt[i] = bt[j];
                bt[j] = temp;
                temp = p[i];
                p[i] = p[j];
                p[j] = temp;
            }
        }
    }
    wt[0] = 0;
    for(i = 1; i < n; i++) {
        wt[i] = wt[i - 1] + bt[i - 1];
    }
    for(i = 0; i < n; i++) {
        tat[i] = wt[i] + bt[i];
    }
    float avg_wt = 0, avg_tat = 0;
    printf("\nProcess\tPriority\tBurst Time\tWaiting Time\tTurnaround Time\n");
```

```

for(i = 0; i < n; i++) {
    printf("P%d\t%d\t%d\t%d\t%d\t%d\n",
        p[i], pr[i], bt[i], wt[i], tat[i]);
    avg_wt += wt[i];
    avg_tat += tat[i];
}
avg_wt /= n;
avg_tat /= n;
printf("\nAverage Waiting Time = %.2f", avg_wt);
printf("\nAverage Turnaround Time = %.2f\n", avg_tat);
return 0;
}

```

Output:

```

Default: "C:\Users\anany\Desktop\OS\Priority
scheduling.exe"
ctrl+alt+1

Enter number of processes: 4
Enter Burst Time and Priority:
P1 Burst Time: 10
P1 Priority: 3
P2 Burst Time: 5
P2 Priority: 1
P3 Burst Time: 8
P3 Priority: 4
P4 Burst Time: 6
P4 Priority: 2

Process Priority      Burst Time      Waiting Time      Turnaround Time
P2         1             5                0                 5
P4         2             6                5                11
P1         3            10               11               21
P3         4             8               21               29

Average Waiting Time = 9.25
Average Turnaround Time = 16.50

Process returned 0 (0x0)   execution time : 21.663 s
Press any key to continue.
|

```

Observation:

Priority Scheduling

```

#include <stdio.h>
#define Process 5
int pid,
int at,
int bt,
int ps,
int ct,
int tot;
int tot;
int bt;
int at;
int dt;
int done;

void calculateSum(Process p[], int n) {
    float total_at = 0, total_bt = 0;
    printf("Enter AT, BT, CT, PS, DT for each process\n");
    for (i = 0; i < n; i++) {
        printf("Process %d: AT: %d, BT: %d, CT: %d, PS: %d, DT: %d\n", i+1, at[i], bt[i], ct[i], ps[i], dt[i]);
        total_at += at[i];
        total_bt += bt[i];
    }
    printf("Average AT: %.2f ms\n", total_at/n);
    printf("Average BT: %.2f ms\n", total_bt/n);
    printf("Enter choice 2\n");

```

```

while (completed < n) {
    int i = 0;
    int highest_ps = 0;
    for (i = 0; i < n; i++) {
        if (ps[i] < highest_ps) {
            highest_ps = ps[i];
            idx = i;
        }
    }
    if (idx != -1) {
        time = p[idx].bt;
        p[idx].ct = time;
        p[idx].done++;
        completed++;
        if (p[idx].ps < highest_ps) {
            highest_ps = p[idx].ps;
            idx = i;
        }
    }
}

if (idx != -1) {
    time = p[idx].bt;
    p[idx].ct = time;
    p[idx].done++;
    completed++;
    if (p[idx].ps < highest_ps) {
        highest_ps = p[idx].ps;
        idx = i;
    }
}

void preemptive (struct Process p[], int n) {
    int completed = 0, time = 0;
    while (completed < n) {
        int idx = -1;
        int highest_ps = 0;
        for (int i = 0; i < n; i++) {
            if (p[i].at < time || (p[i].at == time && p[i].ps > highest_ps)) {
                highest_ps = p[i].ps;
                idx = i;
            }
        }
        if (idx != -1) {
            time = p[idx].at;
            p[idx].bt = p[idx].bt - p[idx].ct;
            total_at += p[idx].at;
            total_bt += p[idx].bt;
            p[idx].ct = p[idx].bt;
            p[idx].at = p[idx].at + p[idx].bt;
            p[idx].done++;
            completed++;
            if (p[idx].ps < highest_ps) {
                highest_ps = p[idx].ps;
                idx = i;
            }
        }
    }
    printf("Average AT: %.2f ms\n", total_at/n);
    printf("Average BT: %.2f ms\n", total_bt/n);
    printf("Enter choice 2\n");

```

```

if (idx != -1) {
    p[idx].ct = p[idx].bt;
    time = p[idx].bt;
    p[idx].done++;
    completed++;
    if (p[idx].ps < highest_ps) {
        highest_ps = p[idx].ps;
        idx = i;
    }
}

if (idx != -1) {
    time = p[idx].bt;
    p[idx].ct = time;
    p[idx].done++;
    completed++;
    if (p[idx].ps < highest_ps) {
        highest_ps = p[idx].ps;
        idx = i;
    }
}

void preemptive (struct Process p[], int n) {
    int completed = 0, time = 0;
    while (completed < n) {
        int idx = -1;
        int highest_ps = 0;
        for (int i = 0; i < n; i++) {
            if (p[i].at < time || (p[i].at == time && p[i].ps > highest_ps)) {
                highest_ps = p[i].ps;
                idx = i;
            }
        }
        if (idx != -1) {
            time = p[idx].at;
            p[idx].bt = p[idx].bt - p[idx].ct;
            total_at += p[idx].at;
            total_bt += p[idx].bt;
            p[idx].ct = p[idx].bt;
            p[idx].at = p[idx].at + p[idx].bt;
            p[idx].done++;
            completed++;
            if (p[idx].ps < highest_ps) {
                highest_ps = p[idx].ps;
                idx = i;
            }
        }
    }
    printf("Average AT: %.2f ms\n", total_at/n);
    printf("Average BT: %.2f ms\n", total_bt/n);
    printf("Enter choice 2\n");

```

d) Round Robin

```
#include <stdio.h>

int main() {
    int n, i, quantum;
    printf("Enter number of processes: ");
    scanf("%d", &n);
    int bt[n], rem_bt[n], wt[n], tat[n];
    printf("Enter Burst Times:\n");
    for(i = 0; i < n; i++) {
        printf("P%d: ", i + 1);
        scanf("%d", &bt[i]);
        rem_bt[i] = bt[i];
    }
    printf("Enter Time Quantum: ");
    scanf("%d", &quantum);
    int time = 0;
    while(1) {
        int done = 1;
        for(i = 0; i < n; i++) {
            if(rem_bt[i] > 0) {
                done = 0;
                if(rem_bt[i] > quantum) {
                    time += quantum;
                    rem_bt[i] -= quantum;
                }
                else {
                    time += rem_bt[i];
                    wt[i] = time - bt[i];
                    rem_bt[i] = 0;
                }
            }
        }
        if(done)
            break;
    }
    float avg_wt = 0, avg_tat = 0;
    printf("\nProcess\tBurst Time\tWaiting Time\tTurnaround Time\n");
    for(i = 0; i < n; i++) {
        tat[i] = bt[i] + wt[i];
        printf("P%d\t%d\t%d\t%d\n",
            i + 1, bt[i], wt[i], tat[i]);
    }
}
```

```

        avg_wt += wt[i];
        avg_tat += tat[i];
    }
    avg_wt /= n;
    avg_tat /= n;
    printf("\nAverage Waiting Time = %.2f", avg_wt);
    printf("\nAverage Turnaround Time = %.2f\n", avg_tat);
    return 0;
}

```

Output:

```

C:\Users\anany\Desktop\OS\RR.exe
ctrl+alt+1

Enter number of processes: 4
Enter Burst Times:
P1: 10
P2: 5
P3: 8
P4: 6
Enter Time Quantum: 6

Process Burst Time      Waiting Time      Turnaround Time
P1      10              17                27
P2       5               6                 11
P3       8              21                 29
P4       6              17                 23

Average Waiting Time = 15.25
Average Turnaround Time = 22.50

Process returned 0 (0x0)   execution time : 19.402 s
Press any key to continue.
|

```

Observation:

Round Robin

```

#include <stdio.h>
struct Process {
    int pid;
    int at;
    int bt;
    int st;
    int ct;
    int tat;
    int wt;
};

void calculate(struct Process p[], int n) {
    float total_wt = 0, total_tat = 0;
    printf("PID\tAT\tBT\tCT\tTAT\tWT\n");
    for (int i = 0; i < n; i++) {
        p[i].ct = p[i].bt - p[i].at;
        p[i].wt = p[i].tat - p[i].bt;
        total_wt += p[i].wt;
        total_tat += p[i].tat;
        printf("P %d\t%d\t%d\t%d\t%d\t%d\n",
            p[i].pid, p[i].at, p[i].bt,
            p[i].ct, p[i].tat, p[i].wt);
    }
    printf("Average WT = %.2f, total_wt/n);
    printf("Average TAT = %.2f\n", total_tat/n);
}

void RoundRobin(struct Process p[], int n, int tq) {
    int time = 0, completed = 0;
    int queue[100], front = 0, rear = 0;
    int visited[100] = {0};
    for (int i = 0; i < n; i++) {
        if (p[i].at == 0) {
            queue[rear++] = i;
        }
    }

    int main() {
        int n, tq;
        printf("Enter number of processes: ");
        scanf("%d", &n);
        struct Process p[n];
        for (int i = 0; i < n; i++) {
            p[i].pid = i+1;
            printf("Process %d: ", i+1);
            printf("Arrival time: ");
            scanf("%d", &p[i].at);
            printf("Burst Time: ");
            scanf("%d", &p[i].bt);
            p[i].st = p[i].bt;
        }
        printf("\nEnter Time Quantum: ");
        scanf("%d", &tq);
        RoundRobin(p, n, tq);
        calculate(p, n);
        return 0;
    }

```

Output:

Enter number of processes: 5

Process P1
Arrival time = 0
Burst time = 5

Process P2
Arrival time = 1
Burst time = 3

Process P3
Arrival time = 2

```

        visited[i] = 1;
    }
    while (completed < n) {
        if (front == rear) {
            time++;
            for (int i = 0; i < n; i++) {
                if (p[i].at <= time && !visited[i]) {
                    queue[rear++] = i;
                    visited[i] = 1;
                }
            }
            continue;
        }
        int i = queue[front++];
        if (p[i].at > time) {
            time = p[i].at;
            p[i].st = time;
            completed++;
        }
        for (int j = 0; j < n; j++) {
            if (p[j].at <= time && !visited[j]) {
                queue[rear++] = j;
                visited[j] = 1;
            }
        }
        if (p[i].st > 0) {
            Burst time = 1
            Process P4
            Arrival time = 3
            Burst time = 2

            Process P5
            Arrival time = 4
            Burst time = 3
            Enter time quantum = 2

            Process AT BT CT TAT WT
            P1 0 5 13 13 8
            P2 1 3 12 11 8
            P3 2 1 5 3 2
            P4 3 2 9 6 4
            P5 4 3 14 10 7

            P1 P2 P3 P4 P5 P2 P1 P5
            0 1 2 3 4 5 6 7 8 9 10 11 12 13 14

            Average WT = 5.8 ms
            Average TAT = 8.6 ms
        }
    }

```

Lab program 2:

Write a C program to simulate a multi-level queue scheduling algorithm considering the following scenario. All the processes in the system are divided into two categories – system processes and user processes. System processes are to be given higher priority than user processes. Use FCFS scheduling for the processes in each queue.

```
#include <stdio.h>

struct Process {
    int id;
    int burst_time;
    int waiting_time;
    int turnaround_time;
};

void fcfs(struct Process p[], int n) {
    int i;
    p[0].waiting_time = 0;
    for (i = 1; i < n; i++) {
        p[i].waiting_time = p[i - 1].waiting_time + p[i - 1].burst_time;
    }
    for (i = 0; i < n; i++) {
        p[i].turnaround_time = p[i].waiting_time + p[i].burst_time;
    }
}

void display(struct Process p[], int n, char name[]) {
    int i;
    float total_wt = 0, total_tat = 0;
    printf("\n%s Queue:\n", name);
    printf("Process\tBurst Time\tWaiting Time\tTurnaround Time\n");
    for (i = 0; i < n; i++) {
        printf("P%d\t%d\t%d\t%d\n",
            p[i].id,
            p[i].burst_time,
            p[i].waiting_time,
            p[i].turnaround_time);
        total_wt += p[i].waiting_time;
        total_tat += p[i].turnaround_time;
    }
    printf("Average Waiting Time = %.2f\n", total_wt / n);
    printf("Average Turnaround Time = %.2f\n", total_tat / n);
}

int main() {
```

```

int n_sys, n_user, i;
printf("Enter number of System Processes: ");
scanf("%d", &n_sys);
struct Process sys[n_sys];
for (i = 0; i < n_sys; i++) {
    sys[i].id = i + 1;
    printf("Enter burst time for System Process P%d: ", sys[i].id);
    scanf("%d", &sys[i].burst_time);
}
printf("\nEnter number of User Processes: ");
scanf("%d", &n_user);
struct Process user[n_user];
for (i = 0; i < n_user; i++) {
    user[i].id = i + 1;
    printf("Enter burst time for User Process P%d: ", user[i].id);
    scanf("%d", &user[i].burst_time);
}
fcfs(sys, n_sys);
fcfs(user, n_user);
int total_sys_time = 0;
for (i = 0; i < n_sys; i++) {
    total_sys_time += sys[i].burst_time;
}
for (i = 0; i < n_user; i++) {
    user[i].waiting_time += total_sys_time;
    user[i].turnaround_time = user[i].waiting_time + user[i].burst_time;
}
printf("\n===== Multi-Level Queue Scheduling =====\n");
display(sys, n_sys, "System (Higher Priority)");
display(user, n_user, "User (Lower Priority)");
return 0;
}

```

Output:

```
Default: "C:\Users\RMSCECSE-14\Desktop
\BM24-41\MultiLevel Queueing.exe"
ctrl+alt+f

" C:\Users\RMSCECSE-14\Desk  X  +  v  Open with  -  □  X

Enter number of System Processes: 4
Enter burst time for System Process P1: 4
Enter burst time for System Process P2: 5
Enter burst time for System Process P3: 3
Enter burst time for System Process P4: 6

Enter number of User Processes: 4
Enter burst time for User Process P1: 8
Enter burst time for User Process P2: 9
Enter burst time for User Process P3: 5
Enter burst time for User Process P4: 7

===== Multi-Level Queue Scheduling =====

System (Higher Priority) Queue:
Process Burst Time    Waiting Time    Turnaround Time
P1      4              0              4
P2      5              4              9
P3      3              9              12
P4      6             12              18
Average Waiting Time = 6.25
Average Turnaround Time = 10.75

User (Lower Priority) Queue:
Process Burst Time    Waiting Time    Turnaround Time
P1      8             18             26
P2      9             26             35
P3      5             35             40
P4      7             40             47
Average Waiting Time = 29.75
Average Turnaround Time = 37.00

Process returned 0 (0x0)   execution time : 194.541 s
Press any key to continue.
|
```

Observation:

```

Multilevel queuing
#include <stdio.h>
struct Process {
    int id;
    int burst_time;
    int waiting_time;
    int tat;
};

void fcts (struct Process p[], int n) {
    int i;
    p[0].waiting_time = 0;
    for (i = 1; i < n; i++) {
        p[i].waiting_time = p[i-1].waiting_time +
                             p[i-1].burst_time;
    }
    for (i = 0; i < n; i++) {
        p[i].tat = p[i].waiting_time + p[i].burst_time;
    }
}

void display (struct Process p[], int n, char name[]) {
    int i;
    float total_wt = 0, total_tat = 0;
    printf("\n %s Queue: %s\n", name);
    printf("Process | BT | WT | TAT | \n");
    for (i = 0; i < n; i++) {
        printf("P%d | %d | %d | %d | \n",
               p[i].id, p[i].burst_time,
               p[i].waiting_time, p[i].tat);
        total_wt += p[i].waiting_time;
    }
}

```

Output:

Process	Queue	AT	BT
P1	↓	0	4
P2	System Queue	1	5
P3		2	3
P4		3	6

Process	Queue	AT	BT
P1	↓	0	8
P2	User Queue	1	9
P3		2	5
P4		3	7

Enter number of System Processes: 4
 Enter BT for P1: 4
 Enter BT for P2: 5
 Enter BT for P3: 3
 Enter BT for P4: 6

Enter number of User Processes: 4
 Enter BT for P1: 8
 Enter BT for P2: 9
 Enter BT for P3: 5
 Enter BT for P4: 7

Multilevel queue Scheduling

System (Higher Priority) Queue:

```

int main() {
    int n_sys, n_user, i;
    printf("Enter no of System Processes: ");
    scanf("%d", &n_sys);
    struct Process user[n_user];
    for (i = 0; i < n_user; i++) {
        user[i].id = i+1;
        printf("Enter burst time for User Process\n Process P%d:", user[i].id);
        scanf("%d", &user[i].burst_time);
    }
    fcts(sys, n_sys);
    fcts(user, n_user);
    int total_sys_time = 0;
    for (i = 0; i < n_sys; i++) {
        total_sys_time += sys[i].burst_time;
    }
    for (i = 0; i < n_user; i++) {
        user[i].waiting_time += total_sys_time;
        user[i].tat = user[i].waiting_time +
                      user[i].burst_time;
    }
    printf("Multilevel Queue Scheduling");
    display(sys, n_sys, "System (Higher Priority)");
    display(user, n_user, "User (Lower Priority)");
}

```

Process	BT	WT	TAT
P1	4	0	4
P2	5	4	9
P3	3	9	12
P4	6	12	18

Average Waiting Time = 6.25 ms
 Average Turnaround Time = 10.75 ms

User (Lower Priority) Queue:

Process	BT	WT	TAT
P1	8	18	26
P2	9	26	35
P3	5	35	40
P4	7	40	47

Average Waiting Time = 29.75 ms
 Average Turnaround Time = 37 ms

P1	P2	P3	P4	P1	P2	P3	P4
0	4	9	12	18	26	35	46

$\frac{P_1}{10/125}$

Lab program 3:

Write a C program to simulate Real-Time CPU Scheduling algorithms:

a) Rate- Monotonic b) Earliest-deadline First c) Proportional scheduling

a) Rate- Monotonic

```
#include <stdio.h>
int main() {
    int n, i, j;
    printf("Enter number of tasks: ");
    scanf("%d", &n);
    int task[n], period[n], burst[n];
    for(i = 0; i < n; i++) {
        task[i] = i + 1;
        printf("\nTask %d Execution Time: ", i + 1);
        scanf("%d", &burst[i]);
        printf("Task %d Period: ", i + 1);
        scanf("%d", &period[i]);
    }
    for(i = 0; i < n - 1; i++) {
        for(j = i + 1; j < n; j++) {
            if(period[i] > period[j]) {
                int temp;
                temp = period[i];
                period[i] = period[j];
                period[j] = temp;
                temp = burst[i];
                burst[i] = burst[j];
                burst[j] = temp;
                temp = task[i];
                task[i] = task[j];
                task[j] = temp;
            }
        }
    }
    printf("\n\nRate Monotonic Scheduling (RMS)\n");
    printf("Task\tExecution Time\tPeriod\tPriority\n");
    for(i = 0; i < n; i++) {
        printf("T%d\t%d\t%d\t%d\n",
            task[i], burst[i], period[i], i + 1);
    }
}
```

```

    }
    printf("\nExecution Sequence:\n");
    for(i = 0; i < n; i++) {
        printf("T%d", task[i]);
        if(i != n - 1)
            printf(" -> ");
    }
    printf("\n");
    return 0;
}

```

Output:

```

C:\Users\anany\Desktop\OS> .\RMS.exe
Enter number of tasks: 4

Task 1 Execution Time: 1
Task 1 Period: 8

Task 2 Execution Time: 2
Task 2 Period: 4

Task 3 Execution Time: 1
Task 3 Period: 12

Task 4 Execution Time: 2
Task 4 Period: 6

Rate Monotonic Scheduling (RMS)
Task   Execution Time  Period  Priority
T2     2              4        1
T4     2              6        2
T1     1              8        3
T3     1             12        4

Execution Sequence:
T2 -> T4 -> T1 -> T3

Process returned 0 (0x0)   execution time : 24.548 s
Press any key to continue.

```

Observation:

```

Rate Monotonic Scheduling
#include<stdio.h>
struct Process
{
    int id, period, burst, remaining, next_arrival;
};

int main()
{
    int n, time, type = 0;
    printf("Enter number of processes: ");
    scanf("%d", &n);
    struct Process p[10];
    for (int i = 0; i < n; i++)
    {
        printf("Process %d (Burst Time, Period): ", i+1);
        scanf("%d %d", &p[i].burst, &p[i].period);
        p[i].remaining = p[i].burst;
        p[i].next_arrival = 0;
        p[i].id = i+1;
        if (p[i].period > type)
            type = p[i].period;
    }

    printf("\n RMS\n");
    printf("Time |< Running |<");
    for (int i = 0; i < n; i++)
        printf(" %d |< ", p[i].id);
    for (time = 0; time < type * 2; time += 1)
    {
        int highest = -1;
        for (int i = 0; i < n; i++)
        {
            if (time == p[i].next_arrival)
            {
                p[i].remaining = p[i].burst;
                p[i].next_arrival = p[i].period;
            }
            if (p[i].remaining > 0)
            {
                if (highest == -1 || p[i].period < p[highest].period)
                    highest = i;
            }
        }
        printf("\n");
        for (int i = 0; i < n; i++)
        {
            if (i == highest)
            {
                printf("%d |< ", p[i].id);
                p[i].remaining--;
            }
            else
                printf(" |< ");
        }
        printf("\n");
    }
}

```

Output:

```

Enter number of processes: 4
Process 1 (Burst Time, Period): 6 8
Process 2 (Burst Time, Period): 4 3
Process 3 (Burst Time, Period): 2 4
Process 4 (Burst Time, Period): 2 3

```

RMS

Time	Running	P1.rem	P2.rem	P3.rem	P4.rem
0	P1	6	4	2	1
1	P1	6	4	2	0
2	P3	6	4	1	0
3	P4	6	4	1	1
4	P4	6	4	1	0
5	P3	6	4	1	0

Gantt chart:

P1	P1	P3	P4	P4	P3	P4	P4	P3	P4
0	1	2	3	4	5	6	7	8	9

b) Earliest-deadline First

```
#include <stdio.h>
int main() {
    int n, i, j;
    printf("Enter number of tasks: ");
    scanf("%d", &n);
    int task[n], burst[n], deadline[n];
    for(i = 0; i < n; i++) {
        task[i] = i + 1;
        printf("\nTask %d Execution Time: ", i + 1);
        scanf("%d", &burst[i]);
        printf("Task %d Deadline: ", i + 1);
        scanf("%d", &deadline[i]);
    }
    for(i = 0; i < n - 1; i++) {
        for(j = i + 1; j < n; j++) {
            if(deadline[i] > deadline[j]) {
                int temp;
                temp = deadline[i];
                deadline[i] = deadline[j];
                deadline[j] = temp;
                temp = burst[i];
                burst[i] = burst[j];
                burst[j] = temp;
                temp = task[i];
                task[i] = task[j];
                task[j] = temp;
            }
        }
    }
    printf("\n\nEarliest Deadline First (EDF) Scheduling\n");
    printf("Task\tExecution Time\tDeadline\tPriority\n");
    for(i = 0; i < n; i++) {
        printf("T%d\t%d\t%d\t%d\n",
            task[i], burst[i], deadline[i], i + 1);
    }
    int currentTime = 0;
    printf("\nDeadline Status:\n");
    printf("Task\tCompletion Time\tStatus\n");
```

```

for(i = 0; i < n; i++) {
    currentTime += burst[i];
    if(currentTime <= deadline[i])
        printf("T%d\t%d\t\tMet\n", task[i], currentTime);
    else
        printf("T%d\t%d\t\tMissed\n", task[i], currentTime);
}
printf("\nExecution Sequence:\n");
for(i = 0; i < n; i++) {
    printf("T%d", task[i]);
    if(i != n - 1)
        printf(" -> ");
}
printf("\n");
return 0;
}

```

Output:

```

C:\Users\janany\Desktop\OS\EDF.exe
Enter number of tasks: 4
Task 1 Execution Time: 2
Task 1 Deadline: 8
Task 2 Execution Time: 1
Task 2 Deadline: 4
Task 3 Execution Time: 3
Task 3 Deadline: 12
Task 4 Execution Time: 2
Task 4 Deadline: 6

Earliest Deadline First (EDF) Scheduling
Task    Execution Time    Deadline    Priority
T2      1                4           1
T4      2                6           2
T1      2                8           3
T3      3                12          4

Deadline Status:
Task    Completion Time    Status
T2      1                Met
T4      3                Met
T1      5                Met
T3      8                Met

Execution Sequence:
T2 -> T4 -> T1 -> T3

Process returned 0 (0x0)   execution time : 20.986 s
Press any key to continue.

```

Observation:

Earliest Deadline First

```
#include <stdio.h>
struct Task {
    int id;
    int capacity;
    int period;
    int deadline;
    int remaining;
};

int main() {
    int n, total_time;
    printf("Enter number of tasks:");
    scanf("%d", &n);
    Task tasks[n];

    for (int i = 0; i < n; i++) {
        tasks[i].id = i+1;
        printf("Enter Capacity and Period for Task %d:", i+1);
        scanf("%d %d", &tasks[i].capacity, &tasks[i].period);
        tasks[i].deadline = tasks[i].period;
        tasks[i].remaining = tasks[i].capacity;
    }

    printf("Enter total simulation time:");
    scanf("%d", &total_time);
    printf("\n Scheduling Log \n");
    for (int t = 0; t < total_time; t++) {
        int selected_task = -1;
        int min_deadline = 9999;

        for (int i = 0; i < n; i++) {
            if (tasks[i].remaining > 0) {
                if (tasks[i].deadline < min_deadline) {
                    min_deadline = tasks[i].deadline;
                    selected_task = i;
                }
            }
        }

        if (selected_task != -1) {
            printf("Time %d: Task %d is running\n", t, selected_task);
            tasks[selected_task].remaining--;
            if (tasks[selected_task].remaining == 0) {
                tasks[selected_task].deadline = tasks[selected_task].period;
                tasks[selected_task].remaining = tasks[selected_task].capacity;
            }
        } else {
            printf("Time %d: CPU Idle\n", t);
        }
    }

    return 0;
}
```

OUTPUT

```
Enter number of processes: 3
Process 1: 1 4
Process 2: 2 6
Process 3: 3 8

PID    Burst time    Period
P1      1             4
P2      2             6
P3      3             8
```

Scheduling Log

Time	Scheduling
0	P1
1	P2
2	P3
3	P1
4	P1
5	Idle
6	P2
7	P3
8	P1
9	P3
10	Idle
11	P3

c) Proportional scheduling

```
#include <stdio.h>

int main() {
    int n, i;
    float totalWeight = 0, totalCPU;
    printf("Enter number of tasks: ");
    scanf("%d", &n);
    int task[n];
    float weight[n], cpuShare[n];
    printf("Enter Total CPU Time Available: ");
    scanf("%f", &totalCPU);
    for(i = 0; i < n; i++) {
        task[i] = i + 1;
        printf("\nEnter weight for Task %d: ", i + 1);
        scanf("%f", &weight[i]);
        totalWeight += weight[i];
    }
    for(i = 0; i < n; i++) {
        cpuShare[i] = (weight[i] / totalWeight) * totalCPU;
    }
    printf("\n\nProportional Scheduling\n");
    printf("Task\tWeight\t\tCPU Share\n");
    for(i = 0; i < n; i++) {
        printf("T%d\t%.2f\t\t%.2f\n",
            task[i], weight[i], cpuShare[i]);
    }
    printf("\nExecution Sequence:\n");
    for(i = 0; i < n; i++) {
        printf("T%d", task[i]);
        if(i != n - 1)
            printf(" -> ");
    }
    printf("\n");
    return 0;
}
```

Output:

```
Default: C:\Users\anany\Desktop\OS\prop-sch.exe
ctrl+alt+f

C:\Users\anany\Desktop\OS\>

Enter number of tasks: 3
Enter Total CPU Time Available: 100

Enter weight for Task 1: 50
Enter weight for Task 2: 30
Enter weight for Task 3: 20

Proportional Scheduling
Task   Weight   CPU Share
T1     50.00    50.00
T2     30.00    30.00
T3     20.00    20.00

Execution Sequence:
T1 -> T2 -> T3
```

Observation:

Proportional Sharing

```
#include <iostream>
int main() {
    int n, i;
    int burst[10], share[10];
    float cputime[10], totalBurst = 0;
    printf("Enter number of processes: ");
    scanf("%d", &n);
    for (i = 0; i < n; i++) {
        printf("\n Process P%d \n", i+1);
        printf("Enter Burst Time: ");
        scanf("%d", &burst[i]);
        printf("Enter CPU Share (in %): ");
        scanf("%d", &share[i]);
        totalBurst += burst[i];
    }

    printf("\nProportional Sharing Scheduling \n");
    printf("Process | Burst Time | CPU Share | Allocated CPU Time \n");
    for (i = 0; i < n; i++) {
        cputime[i] = (share[i] / 100.0) * totalBurst;
        printf("P%d | %d | %d% | %d \n", i+1, burst[i], share[i], cputime[i]);
    }

    return 0;
}
```

OUTPUT:

Enter number of processes: 4

Process P1

Enter Burst Time: 10

Enter CPU Share (in %): 30

Process P2

Enter Burst Time: 15

Enter CPU Share (in %): 30

Process P3

Enter Burst Time: 20

Enter CPU Share (in %): 25

Process P4

Enter Burst Time: 5

Enter CPU Share (in %): 25

Proportional Sharing Scheduling

Process	Burst Time	CPU Share	Allocated CPU Time
P1	10	30%	10.00
P2	15	30%	15.00
P3	20	25%	12.50
P4	5	25%	12.50

Total Burst Time = 10 + 15 + 20 + 5 = 50

$P_1 \rightarrow \frac{30}{100} \times 50 = 15$

$P_2 \rightarrow \frac{30}{100} \times 50 = 15$

$P_3 \rightarrow \frac{25}{100} \times 50 = 12.5$

$P_4 \rightarrow \frac{25}{100} \times 50 = 12.5$

Lab program 4:

Write a C program to simulate:

a) Producer-Consumer problem using semaphores. b) Dining-Philosopher's problem

a) Producer-Consumer problem using semaphores

```
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>
#define BUFFER_SIZE 8
int buffer[BUFFER_SIZE];
int in = 0;
int out = 0;
int totalItems;
sem_t empty;
sem_t full;
pthread_mutex_t mutex;
int produceItem(int item)
{
    return item;
}
void consumeItem(int item)
{
    printf("Consumed Item: %d\n", item);
}
void* producer(void* arg)
{
    int item;
    for (int i = 1; i <= totalItems; i++)
    {
        item = produceItem(i);
        sem_wait(&empty);
        pthread_mutex_lock(&mutex);
        buffer[in] = item;
        printf("Producer produced %d at position %d\n", item, in);
        in = (in + 1) % BUFFER_SIZE;
        pthread_mutex_unlock(&mutex);
        sem_post(&full);
        sleep(1);
    }
}
```

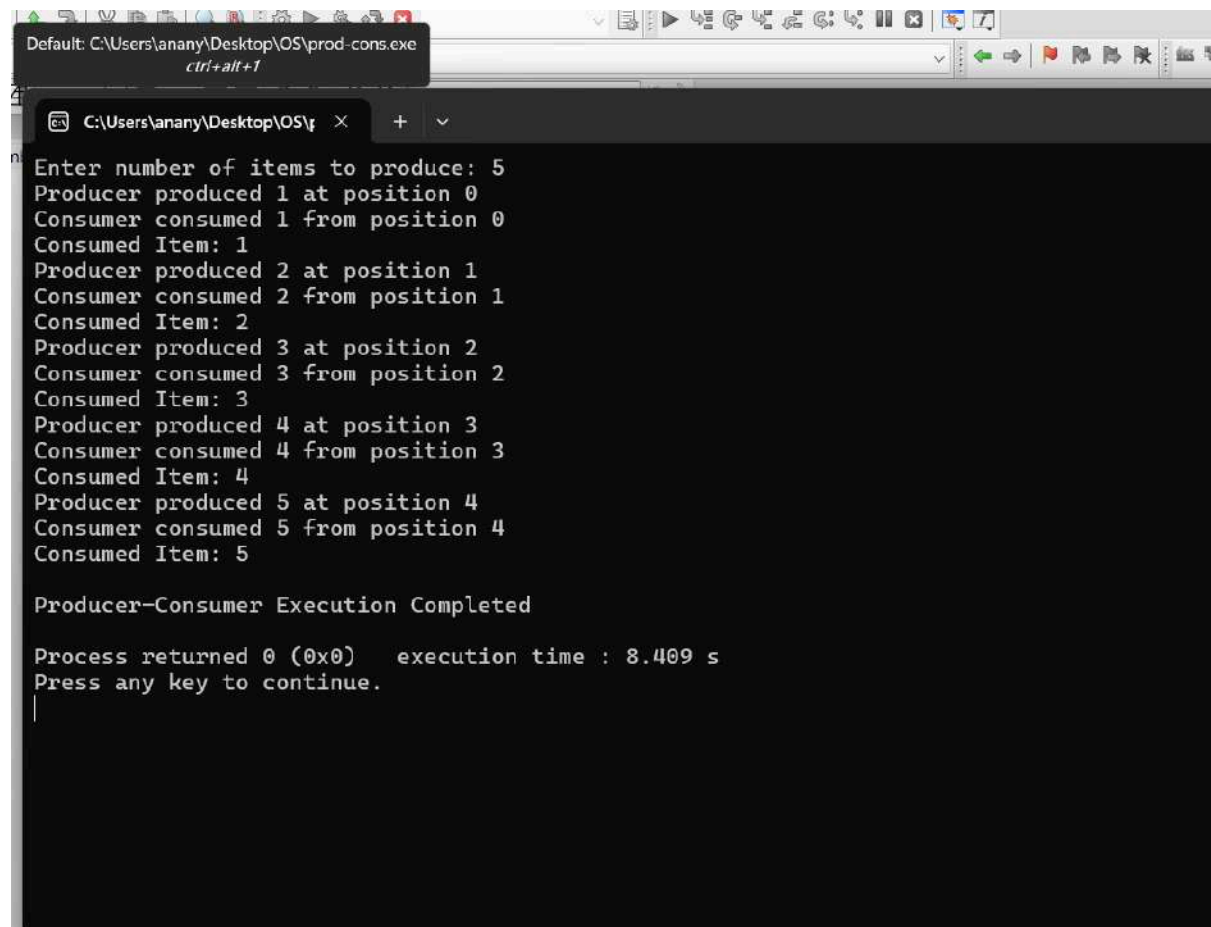
```

    pthread_exit(NULL);
}
void* consumer(void* arg)
{
    int item;
    for (int i = 1; i <= totalItems; i++)
    {
        sem_wait(&full);
        pthread_mutex_lock(&mutex);
        item = buffer[out];
        printf("Consumer consumed %d from position %d\n", item, out);
        out = (out + 1) % BUFFER_SIZE;
        pthread_mutex_unlock(&mutex);
        sem_post(&empty);
        consumeItem(item);
        sleep(1);
    }
    pthread_exit(NULL);
}
void initialize()
{
    sem_init(&empty, 0, BUFFER_SIZE);
    sem_init(&full, 0, 0);
    pthread_mutex_init(&mutex, NULL);
}
void destroy()
{
    sem_destroy(&empty);
    sem_destroy(&full);
    pthread_mutex_destroy(&mutex);
}
int main()
{
    pthread_t producerThread;
    pthread_t consumerThread;
    printf("Enter number of items to produce: ");
    scanf("%d", &totalItems);
    initialize();
    pthread_create(&producerThread, NULL, producer, NULL);
    pthread_create(&consumerThread, NULL, consumer, NULL);

```

```
pthread_join(producerThread, NULL);  
pthread_join(consumerThread, NULL);  
destroy();  
printf("\nProducer-Consumer Execution Completed\n");  
return 0;  
}
```

Output:



```
Default: C:\Users\anany\Desktop\OS\prod-cons.exe  
ctrl+alt+1  
C:\Users\anany\Desktop\OS\>Enter number of items to produce: 5  
Producer produced 1 at position 0  
Consumer consumed 1 from position 0  
Consumed Item: 1  
Producer produced 2 at position 1  
Consumer consumed 2 from position 1  
Consumed Item: 2  
Producer produced 3 at position 2  
Consumer consumed 3 from position 2  
Consumed Item: 3  
Producer produced 4 at position 3  
Consumer consumed 4 from position 3  
Consumed Item: 4  
Producer produced 5 at position 4  
Consumer consumed 5 from position 4  
Consumed Item: 5  
  
Producer-Consumer Execution Completed  
  
Process returned 0 (0x0)   execution time : 8.409 s  
Press any key to continue.  
|
```

Observation:

```

Producers Consumers

#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>

#define BUFFER_SIZE 8

int buffer[BUFFER_SIZE];
int in = 0;
int out = 0;
int totalItems;
sem_t empty;
sem_t full;
pthread_mutex_t mutex;

int produceItem(int item) {
    return item;
}

void consumeItem(int item) {
    printf("Consumed Item: %d\n", item);
}

void * producer(void * arg) {
    int item;
    for (int i = 1; i <= totalItems; i++) {
        item = produceItem(i);
        sem_wait(&empty);
        pthread_mutex_lock(&mutex);
        buffer[in] = item;
        printf("Producer produced %d at position %d\n", item, in);
        in = (in + 1) % BUFFER_SIZE;
        pthread_mutex_unlock(&mutex);
        sem_post(&full);
        consumeItem(item);
        sleep(1);
    }
    pthread_exit(NULL);
}

void initialize() {
    sem_init(&empty, 0, BUFFER_SIZE);
    sem_init(&full, 0, 0);
    pthread_mutex_init(&mutex, NULL);
}

void destroy() {
    sem_destroy(&empty);
    sem_destroy(&full);
    pthread_mutex_destroy(&mutex);
}

int main() {
    pthread_t producerThread;
    pthread_t consumerThread;
    printf("Enter number of items to produce: ");
    scanf("%d", &totalItems);
    initialize();
    pthread_create(&producerThread, NULL, producer, NULL);
    pthread_create(&consumerThread, NULL, consumeItem, NULL);
    pthread_join(producerThread, NULL);
    pthread_join(consumerThread, NULL);
    destroy();
    printf("\n Producers - Consumers Execution Completed\n");
    return 0;
}

OUTPUT:
Enter number of items to produce: 32
Producer produced 1 at position 0
Consumer consumed 1 from position 0
Consumed Item: 1
Producer produced 2 at position 1
Consumer consumed 2 from position 1
Consumed Item: 2

```

b) Dining-Philosopher's problem

```
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>
#define MAX 10
int n;
int state[MAX]; // 0 = THINKING, 1 = HUNGRY, 2 = EATING
pthread_mutex_t mutex;
pthread_cond_t cond[MAX];
#define THINKING 0
#define HUNGRY 1
#define EATING 2
int LEFT(int i) { return (i + n - 1) % n; }
int RIGHT(int i) { return (i + 1) % n; }
void test(int i) {
    if (state[i] == HUNGRY &&
        state[LEFT(i)] != EATING &&
        state[RIGHT(i)] != EATING) {
        state[i] = EATING;
        printf("Philosopher %d is EATING\n", i);
        pthread_cond_signal(&cond[i]);
    }
}
void take_fork(int i) {
    pthread_mutex_lock(&mutex);
    state[i] = HUNGRY;
    printf("Philosopher %d is HUNGRY\n", i);
    test(i);
    while (state[i] != EATING)
        pthread_cond_wait(&cond[i], &mutex);
    pthread_mutex_unlock(&mutex);
}
void put_fork(int i) {
    pthread_mutex_lock(&mutex);
    state[i] = THINKING;
    printf("Philosopher %d is THINKING\n", i);
    test(LEFT(i));
    test(RIGHT(i));
    pthread_mutex_unlock(&mutex);
}
```

```

void* philosopher(void* num) {
    int id = *(int*)num;
    while (1) {
        printf("Philosopher %d is THINKING\n", id);
        sleep(1);
        take_fork(id);
        sleep(2);
        put_fork(id);
    }
}

int main() {
    pthread_t p[MAX];
    int ids[MAX];
    printf("Enter number of philosophers: ");
    scanf("%d", &n);
    pthread_mutex_init(&mutex, NULL);
    for (int i = 0; i < n; i++) {
        pthread_cond_init(&cond[i], NULL);
        state[i] = THINKING;
    }
    for (int i = 0; i < n; i++) {
        ids[i] = i;
        pthread_create(&p[i], NULL, philosopher, &ids[i]);
    }
    for (int i = 0; i < n; i++)
        pthread_join(p[i], NULL);
    return 0;
}

```

Output:

```

PS C:\Users\BMSCECSE-L4\Desktop\BM24-41> C:\Users\BMSCECSE-L4\Desktop\BM24-41\din-phil.exe
Enter number of philosophers: 3
Philosopher 1 is THINKING
Philosopher 0 is THINKING
Philosopher 2 is THINKING
Philosopher 0 is HUNGRY
Philosopher 0 is EATING
Philosopher 2 is HUNGRY
Philosopher 1 is HUNGRY
Philosopher 0 is THINKING
Philosopher 2 is EATING
Philosopher 0 is THINKING
Philosopher 0 is HUNGRY
Philosopher 2 is THINKING
Philosopher 1 is EATING
Philosopher 2 is THINKING
Philosopher 2 is HUNGRY
Philosopher 1 is THINKING
Philosopher 0 is EATING
Philosopher 1 is THINKING
Philosopher 1 is HUNGRY

```

Observation:

File	Page	File	Page
Dining Philosophers			
<pre>#include <stdio.h> #include <pthread.h> #include <unistd.h> #define MAX 30 int n; int state[MAX]; pthread_mutex_t mutex; pthread_cond_t cond[MAX]; #define THINKING 0 #define HUNGRY 1 #define EATING 2 int LEFT(int i) { return (i-1) % n; } int RIGHT(int i) { return (i+1) % n; } void test(int i) { if (state[i] == HUNGRY state[LEFT(i)] == EATING state[RIGHT(i)] == EATING) { state[i] = EATING; printf("Philosopher %d is EATING\n", i); pthread_cond_signal(&cond[i]); } } void take_chopstick(int i) { pthread_mutex_lock(&mutex); state[i] = HUNGRY; printf("Philosopher %d is HUNGRY\n", i); test(i); while (state[i] != EATING) {} }</pre>		<pre>pthread_cond_wait(&cond[i], &mutex); pthread_mutex_unlock(&mutex); } void put_chopstick(int i) { pthread_mutex_lock(&mutex); state[i] = THINKING; printf("Philosopher %d is THINKING\n", i); test(LEFT(i)); test(RIGHT(i)); pthread_mutex_unlock(&mutex); } void *philosopher(void *num) { int id = *(int *) num; while (1) { printf("Philosopher %d is THINKING\n", id); sleep(1); take_chopstick(id); sleep(2); put_chopstick(id); } } int main() { pthread_t p[MAX]; int ids[MAX]; printf("Enter the number of philosophers: "); scanf("%d", &n); pthread_mutex_init(&mutex, NULL); for (int i=0; i<n; i++) { pthread_cond_init(&cond[i], NULL); state[i] = THINKING; ids[i] = i; } }</pre>	<pre>pthread_create(&p[i], NULL, philosopher, &ids[i]); } for (int i=0; i<n; i++) { pthread_join(p[i], NULL); } return 0; }</pre>
			OUTPUT: <pre>Enter number of philosophers: 3 Philosopher 1 is THINKING Philosopher 2 is THINKING Philosopher 0 is THINKING Philosopher 2 is HUNGRY Philosopher 2 is EATING Philosopher 0 is HUNGRY Philosopher 0 is HUNGRY Philosopher 2 is THINKING Philosopher 1 is EATING Philosopher 3 is THINKING Philosopher 2 is HUNGRY</pre>

Lab program 5:

Write a C program to simulate:

a) Bankers' algorithm for the purpose of deadlock avoidance. b) Deadlock Detection

a) Bankers' algorithm for the purpose of deadlock avoidance.

```
#include <stdio.h>
int main()
{
    int n, m, i, j, k;
    printf("Enter number of processes and resources: ");
    scanf("%d %d", &n, &m);
    int alloc[n][m], max[n][m], need[n][m];
    int avail[m], finish[n], safe[n];
    printf("Enter Allocation Matrix:\n");
    for(i = 0; i < n; i++)
        for(j = 0; j < m; j++)
            scanf("%d", &alloc[i][j]);
    printf("Enter Maximum Matrix:\n");
    for(i = 0; i < n; i++)
        for(j = 0; j < m; j++)
            scanf("%d", &max[i][j]);
    printf("Enter Available Resources:\n");
    for(i = 0; i < m; i++)
        scanf("%d", &avail[i]);
    for(i = 0; i < n; i++)
        for(j = 0; j < m; j++)
            need[i][j] = max[i][j] - alloc[i][j];
    for(i = 0; i < n; i++)
        finish[i] = 0;
    int count = 0;
    while(count < n)
    {
        int found = 0;
        for(i = 0; i < n; i++)
        {
            if(finish[i] == 0)
            {
                for(j = 0; j < m; j++)
                    if(need[i][j] > avail[j])
                        break;
            }
        }
        if(found == 0)
            continue;
        for(j = 0; j < m; j++)
            avail[j] -= alloc[i][j];
        finish[i] = 1;
        count++;
    }
}
```

```

        if(j == m)
        {
            for(k = 0; k < m; k++)
                avail[k] += alloc[i][k];
            safe[count++] = i;
            finish[i] = 1;
            found = 1;
        }
    }
}
if(found == 0)
{
    printf("System is NOT in safe state\n");
    return 0;
}
printf("System is in SAFE state\nSafe Sequence: ");
for(i = 0; i < n; i++)
    printf("P%d ", safe[i]);
return 0;
}

```

Output:

```

PS C:\Users\BMSCECSE-L4\Desktop\BM24-41> C:\Users\BMSCECSE-L4\Desktop\BM24-41\bankers.exe
Enter number of processes and resources: 3 2
Enter Allocation Matrix:
0 1
2 0
1 1
Enter Maximum Matrix:
1 2
2 1
3 2
Enter Available Resources:
1 1
System is in SAFE state
Safe Sequence: P0 P1 P2
PS C:\Users\BMSCECSE-L4\Desktop\BM24-41> █

```

Observation:

```
Banker's Algorithm (Deadlock)

#include <stdio.h>
int main() {
    int n, m, i, j, k;
    printf("Enter number of processes and resources\n");
    scanf("%d %d", &n, &m);
    int alloc[n][m], max[n][m], need[n][m];
    int avail[m], finish[n], safe[n];
    printf("Enter Allocation Matrix :\n");
    for (i=0; i<n; i++)
        for (j=0; j<m; j++)
            scanf("%d", &alloc[i][j]);
    printf("Enter Maximum Matrix :\n");
    for (i=0; i<n; i++)
        for (j=0; j<m; j++)
            scanf("%d", &max[i][j]);
    printf("Enter Available Resource :\n");
    for (i=0; i<m; i++)
        scanf("%d", &avail[i]);
    for (i=0; i<n; i++)
        finish[i] = 0;
    int count = 0;
    while (count < n) {
        int found = 0;
        for (i=0; i<n; i++) {
            if (finish[i] == 0) {
                for (j=0; j<m; j++) {
                    if (need[i][j] > avail[j])
                        break;
                }
                if (j == m) {
                    for (k=0; k<m; k++)
                        avail[k] += alloc[i][k];
                    finish[i] = 1;
                    count++;
                }
            }
        }
        if (found == 0) {
            printf("System is NOT in safe state\n");
            return 0;
        }
        printf("System is in SAFE state\n Safe Sequence:");
        for (i=0; i<n; i++)
            printf("P%d", safe[i]);
        return 0;
    }
}

OUTPUT:
Enter number of processes and resources: 3 2
Enter Allocation Matrix: 0 1
                        8 0
                        1 1
Enter Maximum Matrix: 1 2
                      2 1
                      3 2
Enter Available Resource: 1 1
System is in safe state.
Safe Sequence: P0 P1 P2
```

b) Deadlock Detection

```
#include <stdio.h>

int main() {
    int n, m;
    // n = number of processes
    // m = number of resource types
    printf("Enter number of processes: ");
    scanf("%d", &n);
    printf("Enter number of resource types: ");
    scanf("%d", &m);
    int alloc[n][m], request[n][m], avail[m];
    int finish[n];
    printf("\nEnter Allocation Matrix:\n");
    for(int i = 0; i < n; i++) {
        for(int j = 0; j < m; j++) {
            scanf("%d", &alloc[i][j]);
        }
    }
    printf("\nEnter Request Matrix:\n");
    for(int i = 0; i < n; i++) {
        for(int j = 0; j < m; j++) {
            scanf("%d", &request[i][j]);
        }
    }
    printf("\nEnter Available Resources:\n");
    for(int i = 0; i < m; i++) {
        scanf("%d", &avail[i]);
    }
    for(int i = 0; i < n; i++) {
        finish[i] = 0;
    }
    int work[m];
    for(int i = 0; i < m; i++) {
        work[i] = avail[i];
    }
    int found;
    do {
        found = 0;
        for(int i = 0; i < n; i++) {
```

```

        if(finish[i] == 0) {
            int canExecute = 1;
            for(int j = 0; j < m; j++) {
                if(request[i][j] > work[j]) {
                    canExecute = 0;
                    break;
                }
            }
            if(canExecute) {
                for(int j = 0; j < m; j++) {
                    work[j] += alloc[i][j];
                }
                finish[i] = 1;
                found = 1;
                printf("\nProcess P%d executed successfully.", i);
            }
        }
    }
} while(found);
int deadlock = 0;
printf("\n\nDeadlocked Processes: ");
for(int i = 0; i < n; i++) {
    if(finish[i] == 0) {
        printf("P%d ", i);
        deadlock = 1;
    }
}
if(deadlock == 0) {
    printf("None");
    printf("\nSystem is NOT in deadlock.");
} else {
    printf("\nSystem IS in deadlock.");
}
return 0;
}

```

Output:

```
PS C:\Users\BMSCECSE-L4\Desktop\BM24-41> C:\Users\BMSCECSE-L4\Desktop\BM24-41\deadlock-dection.exe
Enter number of processes: 5
Enter number of resource types: 3

Enter Allocation Matrix:
0 1 0
2 0 0
3 0 3
2 1 1
0 0 2

Enter Request Matrix:
0 0 0
2 0 2
0 0 0
1 0 0
0 0 2

Enter Available Resources:
0 0 0

Process P0 executed successfully.
Process P2 executed successfully.
Process P3 executed successfully.
Process P4 executed successfully.
Process P1 executed successfully.

Deadlocked Processes: None
System is NOT in deadlock.
PS C:\Users\BMSCECSE-L4\Desktop\BM24-41> 
```

Observation:

Deadlock Detection

```

#include <stdio.h>
int main() {
    int m, n;
    // n -> no. of processes
    // m -> no. of resource types
    printf("Enter number of processes: ");
    scanf("%d", &n);
    printf("Enter number of resource types: ");
    scanf("%d", &m);
    int alloc[n][m], request[n][m], avail[m];
    int finish[n];
    printf("\n Enter Allocation Matrix: \n");
    for (int i=0; i<n; i++) {
        for (int j=0; j<m; j++) {
            scanf("%d", &alloc[i][j]);
        }
    }
    printf("\n Enter Request Matrix: \n");
    for (int i=0; i<n; i++) {
        for (int j=0; j<m; j++) {
            scanf("%d", &request[i][j]);
        }
    }
    printf("\n Enter Available Resource: \n");
    for (int i=0; i<m; i++) {
        scanf("%d", &avail[i]);
    }
    for (int i=0; i<n; i++) {
        finish[i] = 0;
    }

    if (deadlock == 0) {
        printf("None");
        printf("\n System is NOT in deadlock.");
    } else {
        printf("\n System IS in deadlock.");
    }
    return 0;
}

```

OUTPUT:

Enter number of processes: 5
 Enter number of resource types: 3
 Enter Allocation Matrix:
 0 0 0
 2 0 0
 3 0 3
 2 1 1
 0 0 2

Enter Request Matrix:
 0 0 0
 2 0 2
 0 0 0
 1 0 0
 0 0 2

Enter Available Resources:
 0 0 0

Process P0 executed successfully
 Process P2 executed successfully
~~Deadlock Processes: P1, P3, P4~~
 System IS in deadlock
 Process P3 executed successfully
 Process P4 executed successfully

```

for (int i=0; i<m; i++) {
    work[i] = avail[i];
}
int found;
do {
    found = 0;
    for (int i=0; i<n; i++) {
        if (finish[i] == 0) {
            int countExecute = 0;
            for (int j=0; j<m; j++) {
                if (request[i][j] > work[j]) {
                    countExecute = 0;
                    break;
                }
            }
            if (countExecute) {
                for (int j=0; j<m; j++) {
                    work[j] += alloc[i][j];
                }
                finish[i] = 1;
                found = 1;
                printf("\n Process %d executed successfully.", i);
            }
        }
    }
    while (!found);
    int deadlock = 0;
    printf("\n Deadlock Processes: ");
    for (int i=0; i<n; i++) {
        if (finish[i] == 0) {
            // Deadlock processes
        }
    }
}

```

Process P1 executed successfully
 Deadlock Processes: None
 System is NOT in deadlock.

By 27/5/24

Lab program 6:

Write a C program to simulate the following contiguous memory allocation techniques.

a) Worst-fit b) Best-fit c) First-fit

a) Worst-fit

```
#include <stdio.h>

int main() {
    int nb, np, i, j;
    printf("Enter number of memory blocks: ");
    scanf("%d", &nb);
    int block[nb];
    printf("Enter sizes of memory blocks:\n");
    for(i = 0; i < nb; i++) {
        scanf("%d", &block[i]);
    }
    printf("Enter number of processes: ");
    scanf("%d", &np);
    int process[np], allocation[np];
    printf("Enter sizes of processes:\n");
    for(i = 0; i < np; i++) {
        scanf("%d", &process[i]);
        allocation[i] = -1;
    }
    for(i = 0; i < np; i++) {
        int worstIdx = -1;
        for(j = 0; j < nb; j++) {
            if(block[j] >= process[i]) {
                if(worstIdx == -1 ||
                    block[j] > block[worstIdx]) {
                    worstIdx = j;
                }
            }
        }
        if(worstIdx != -1) {
            allocation[i] = worstIdx;
            block[worstIdx] -= process[i];
        }
    }
    printf("\nWorst Fit Memory Allocation\n");
```

```

printf("Process\tProcess Size\tBlock Allocated\n");
for(i = 0; i < np; i++) {
    printf("P%d\t%d\t", i + 1, process[i]);
    if(allocation[i] != -1)
        printf("%d\n", allocation[i] + 1);
    else
        printf("Not Allocated\n");
}
return 0;
}

```

Output:

```

Default: C:\Users\anany\Desktop\OS\worstfit.exe
ctrl+alt+1

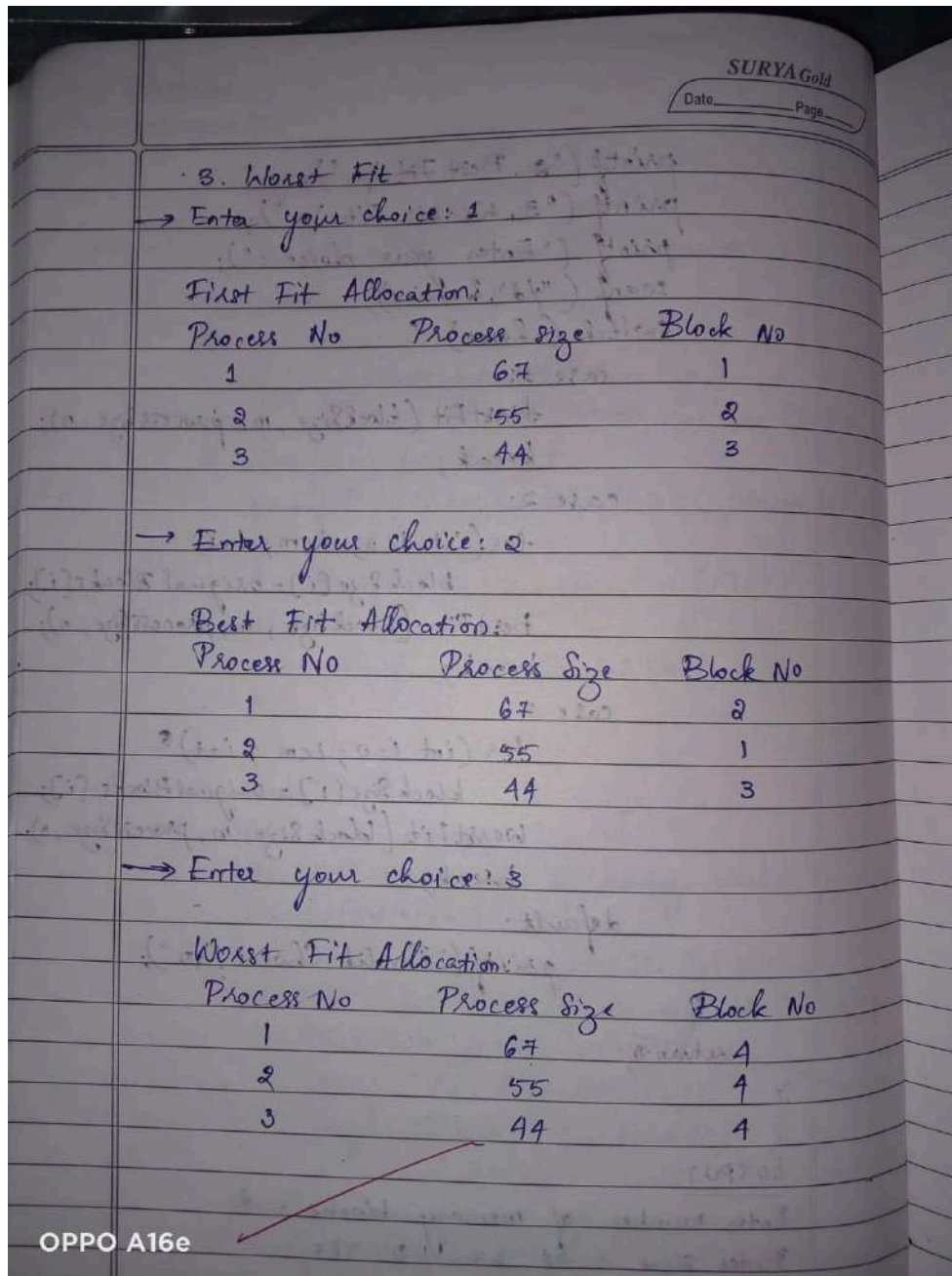
C:\Users\anany\Desktop\OS\ >
Enter number of memory blocks: 5
Enter sizes of memory blocks:
100
500
200
300
600
Enter number of processes: 4
Enter sizes of processes:
212
417
112
426

Worst Fit Memory Allocation
Process Process Size    Block Allocated
P1      212             5
P2      417             2
P3      112             5
P4      426            Not Allocated

Process returned 0 (0x0)   execution time : 27.468 s
Press any key to continue.

```

Observation:

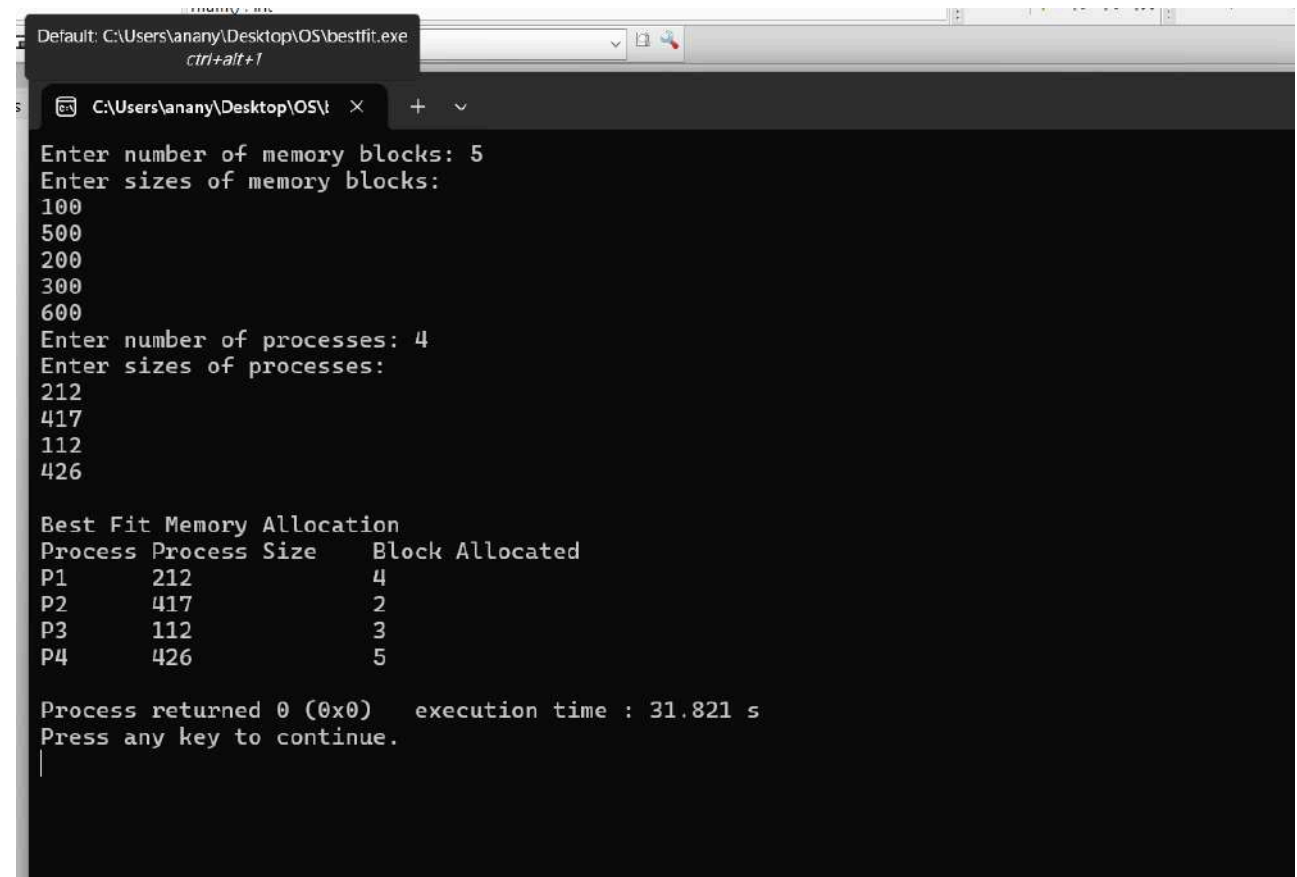


b) Best-fit

```
#include <stdio.h>
int main() {
    int nb, np, i, j;
    printf("Enter number of memory blocks: ");
    scanf("%d", &nb);
    int block[nb];
    printf("Enter sizes of memory blocks:\n");
    for(i = 0; i < nb; i++) {
        scanf("%d", &block[i]);
    }
    printf("Enter number of processes: ");
    scanf("%d", &np);
    int process[np], allocation[np];
    printf("Enter sizes of processes:\n");
    for(i = 0; i < np; i++) {
        scanf("%d", &process[i]);
        allocation[i] = -1;
    }
    for(i = 0; i < np; i++) {
        int bestIdx = -1;
        for(j = 0; j < nb; j++) {
            if(block[j] >= process[i]) {
                if(bestIdx == -1 ||
                    block[j] < block[bestIdx]) {
                    bestIdx = j;
                }
            }
        }
        if(bestIdx != -1) {
            allocation[i] = bestIdx;
            block[bestIdx] -= process[i];
        }
    }
    printf("\nBest Fit Memory Allocation\n");
    printf("Process\tProcess Size\tBlock Allocated\n");
    for(i = 0; i < np; i++) {
        printf("P%d\t%d\t\t", i + 1, process[i]);
        if(allocation[i] != -1)
            printf("%d\n", allocation[i] + 1);
    }
}
```

```
        else
            printf("Not Allocated\n");
    }
    return 0;
}
```

Output:



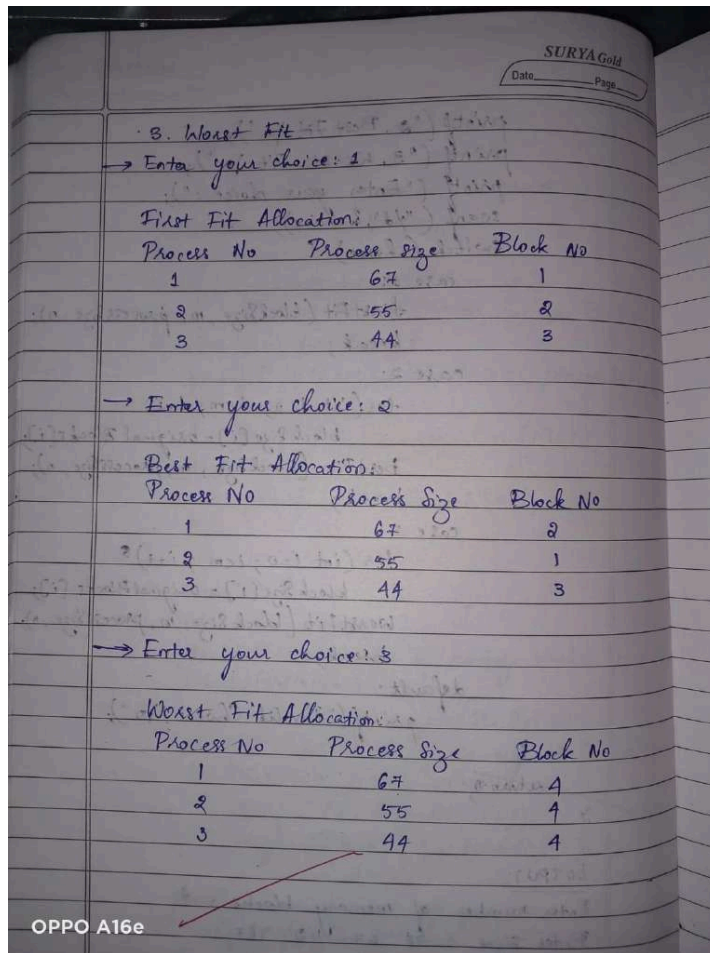
```
Default: C:\Users\anany\Desktop\OS\bestfit.exe
ctrl+alt+l

C:\Users\anany\Desktop\OS\>
Enter number of memory blocks: 5
Enter sizes of memory blocks:
100
500
200
300
600
Enter number of processes: 4
Enter sizes of processes:
212
417
112
426

Best Fit Memory Allocation
Process Process Size    Block Allocated
P1      212             4
P2      417             2
P3      112             3
P4      426             5

Process returned 0 (0x0)   execution time : 31.821 s
Press any key to continue.
|
```

Observation:



c) First-fit

```
#include <stdio.h>

int main() {
    int nb, np, i, j;
    printf("Enter number of memory blocks: ");
    scanf("%d", &nb);
    int block[nb];
    printf("Enter sizes of memory blocks:\n");
    for(i = 0; i < nb; i++) {
        scanf("%d", &block[i]);
    }
    printf("Enter number of processes: ");
    scanf("%d", &np);
    int process[np], allocation[np];
    printf("Enter sizes of processes:\n");
    for(i = 0; i < np; i++) {
        scanf("%d", &process[i]);
        allocation[i] = -1;
    }
    for(i = 0; i < np; i++) {
        for(j = 0; j < nb; j++) {
            if(block[j] >= process[i]) {
                allocation[i] = j;
                block[j] -= process[i];
                break;
            }
        }
    }
    printf("\nFirst Fit Memory Allocation\n");
    printf("Process\tProcess Size\tBlock Allocated\n");
    for(i = 0; i < np; i++) {
        printf("P%d\t%d\t\t", i + 1, process[i]);
        if(allocation[i] != -1)
            printf("%d\n", allocation[i] + 1);
        else
            printf("Not Allocated\n");
    }
    return 0;
}
```

Output:

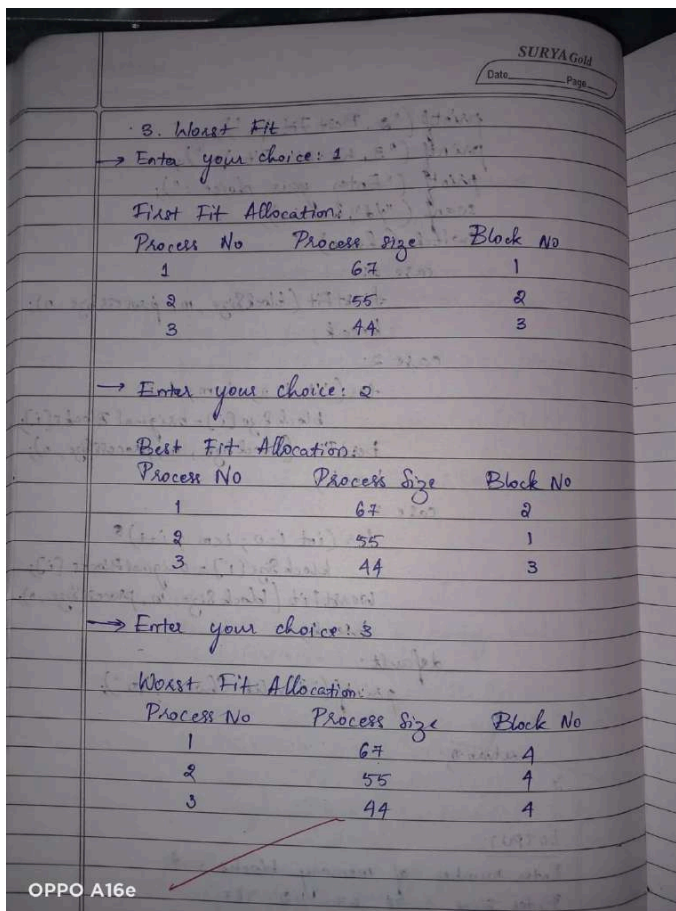
```
Default: C:\Users\anany\Desktop\OS\firstfit.exe
ctrl+alt+f
Untitled3.c  FCF.c  SJF.c  Priority scheduling.c  RR.c  RMS.c  EDF.c  prop-sch.c  prod-cons.c

C:\Users\anany\Desktop\OS\firstfit.exe
Enter number of memory blocks: 5
Enter sizes of memory blocks:
100
500
200
300
600
Enter number of processes: 4
Enter sizes of processes:
212
417
112
426

First Fit Memory Allocation
Process Process Size  Block Allocated
P1      212          2
P2      417          5
P3      112          2
P4      426        Not Allocated

Process returned 0 (0x0)  execution time : 32.439 s
Press any key to continue.
```

Observation:



Lab program 7:

Write a C program to simulate page replacement algorithms.

a) FIFO b) LRU c) Optimal

a) FIFO

```
#include <stdio.h>
int main() {
    int pages, frames, i, j;
    printf("Enter number of pages: ");
    scanf("%d", &pages);
    int page[pages];
    printf("Enter page reference string:\n");
    for(i = 0; i < pages; i++) {
        scanf("%d", &page[i]);
    }
    printf("Enter number of frames: ");
    scanf("%d", &frames);
    int frame[frames];
    for(i = 0; i < frames; i++)
        frame[i] = -1;
    int pageFaults = 0;
    int index = 0;
    printf("\nPage\tFrames\n");
    for(i = 0; i < pages; i++) {
        int found = 0;
        for(j = 0; j < frames; j++) {
            if(frame[j] == page[i]) {
                found = 1;
                break;
            }
        }
        if(!found) {
            frame[index] = page[i];
            index = (index + 1) % frames;
            pageFaults++;
        }
        printf("%d\t", page[i]);
        for(j = 0; j < frames; j++) {
            if(frame[j] != -1)
```

```

        printf("%d ", frame[j]);
    else
        printf("- ");
    }
    printf("\n");
}
printf("Total Page Faults = %d\n", pageFaults);
return 0;
}

```

Output:

```

Default: C:\Users\anany\Desktop\OS\lifo.exe
ctrl+alt+i

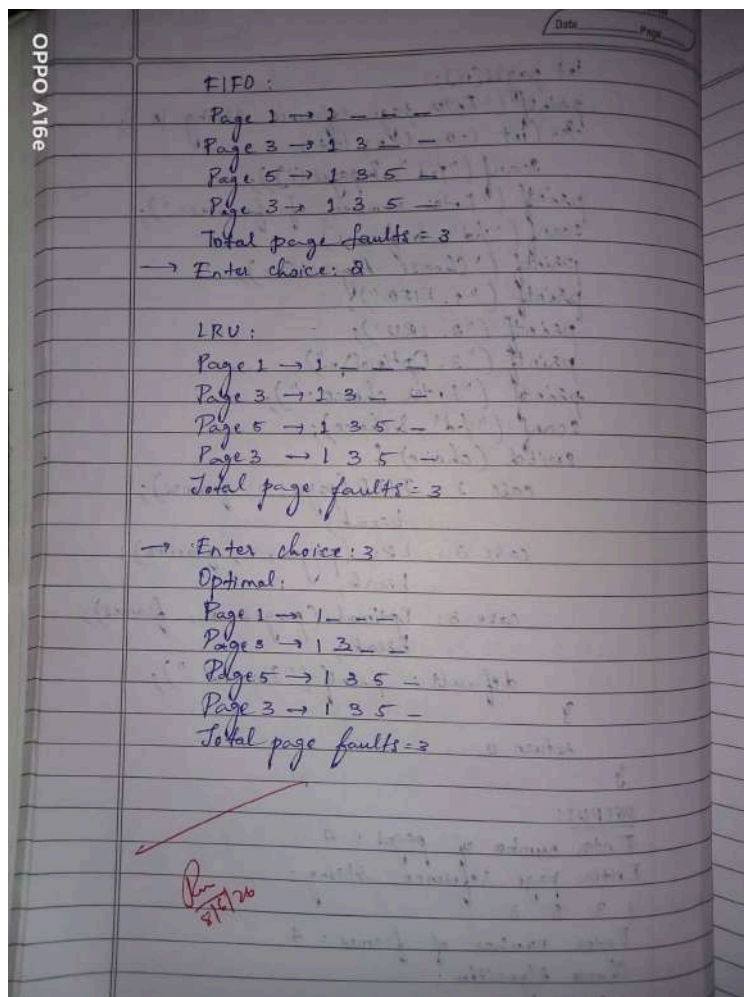
C:\Users\anany\Desktop\OS\lifo.exe
Enter number of pages: 12
Enter page reference string:
1 2 3 4 1 2 5 1 2 3 4 5
Enter number of frames: 3

Page   Frames
1       1 - -
2       1 2 -
3       1 2 3
4       4 2 3
1       4 1 3
2       4 1 2
5       5 1 2
1       5 1 2
2       5 1 2
3       5 3 2
4       5 3 4
5       5 3 4
Total Page Faults = 9

Process returned 0 (0x0)   execution time : 29.079 s
Press any key to continue.

```

Observation:



b) LRU

```
#include <stdio.h>
int main() {
    int n, frames, i, j, k;
    printf("Enter number of pages: ");
    scanf("%d", &n);
    int pages[n];
    printf("Enter page reference string:\n");
    for(i = 0; i < n; i++) {
        scanf("%d", &pages[i]);
    }
    printf("Enter number of frames: ");
    scanf("%d", &frames);
    int frame[frames], recent[frames];
    for(i = 0; i < frames; i++) {
        frame[i] = -1;
        recent[i] = 0;
    }
    int pageFaults = 0, time = 0;
    printf("\nPage\tFrames\n");
    for(i = 0; i < n; i++) {
        int found = 0;
        for(j = 0; j < frames; j++) {
            if(frame[j] == pages[i]) {
                found = 1;
                recent[j] = ++time;
                break;
            }
        }
        if(!found) {
            int lruIndex = 0;
            for(j = 0; j < frames; j++) {
                if(frame[j] == -1) {
                    lruIndex = j;
                    break;
                }
            }
            if(recent[j] < recent[lruIndex])
                lruIndex = j;
        }
    }
}
```

```

        frame[lruIndex] = pages[i];
        recent[lruIndex] = ++time;
        pageFaults++;
    }
    printf("%d\t", pages[i]);
    for(k = 0; k < frames; k++) {
        if(frame[k] != -1)
            printf("%d ", frame[k]);
        else
            printf("- ");
    }
    printf("\n");
}
printf("Total Page Faults = %d\n", pageFaults);
return 0;
}

```

Output:

```

C:\Users\anany\Desktop\OS\lru.exe
ctrl+alt+1

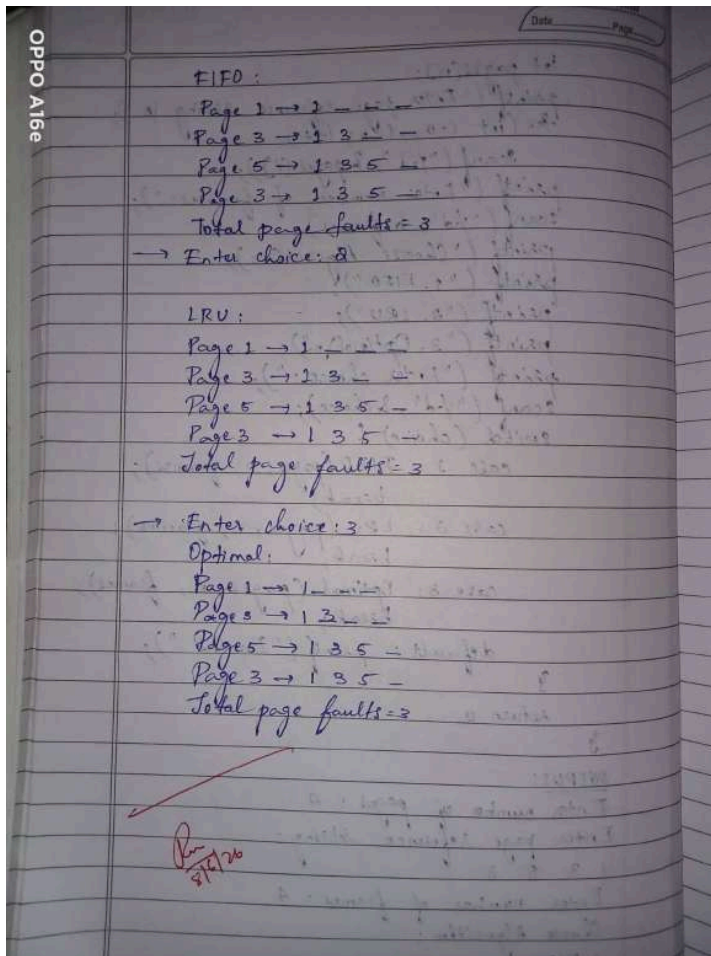
Enter number of pages: 12
Enter page reference string:
1 2 3 4 1 2 5 1 2 3 4 5
Enter number of frames: 3

Page    Frames
1       1 - -
2       1 2 -
3       1 2 3
4       4 2 3
1       4 1 3
2       4 1 2
5       5 1 2
1       5 1 2
2       5 1 2
3       3 1 2
4       3 4 2
5       3 4 5
Total Page Faults = 10

Process returned 0 (0x0)   execution time : 18.594 s
Press any key to continue.
|

```

Observation:



```
#include <stdio.h>

int main() {
    int n, frames, i, j, k;
    printf("Enter number of pages: ");
    scanf("%d", &n);
    int pages[n];
    printf("Enter page reference string:\n");
    for(i = 0; i < n; i++) {
        scanf("%d", &pages[i]);
    }
    printf("Enter number of frames: ");
    scanf("%d", &frames);
    int frame[frames];
    for(i = 0; i < frames; i++)
        frame[i] = -1;
    int pageFaults = 0;
    printf("\nPage\tFrames\n");
    for(i = 0; i < n; i++) {
        int found = 0;
        for(j = 0; j < frames; j++) {
            if(frame[j] == pages[i]) {
                found = 1;
                break;
            }
        }
        if(!found) {
            int replaceIndex = -1;
            for(j = 0; j < frames; j++) {
                if(frame[j] == -1) {
                    replaceIndex = j;
                    break;
                }
            }
            if(replaceIndex == -1) {
                int farthest = -1;
                for(j = 0; j < frames; j++) {
                    int nextUse;
                    for(nextUse = i + 1; nextUse < n; nextUse++) {
                        if(frame[j] == pages[nextUse])
```

```

        break;
    }
    if(nextUse == n) {
        replaceIndex = j;
        break;
    }
    if(nextUse > farthest) {
        farthest = nextUse;
        replaceIndex = j;
    }
}
frame[replaceIndex] = pages[i];
pageFaults++;
}
printf("%d\t", pages[i]);
for(k = 0; k < frames; k++) {
    if(frame[k] != -1)
        printf("%d ", frame[k]);
    else
        printf("- ");
}
printf("\n");
}
printf("Total Page Faults = %d\n", pageFaults);
return 0;
}

```

Output:

```

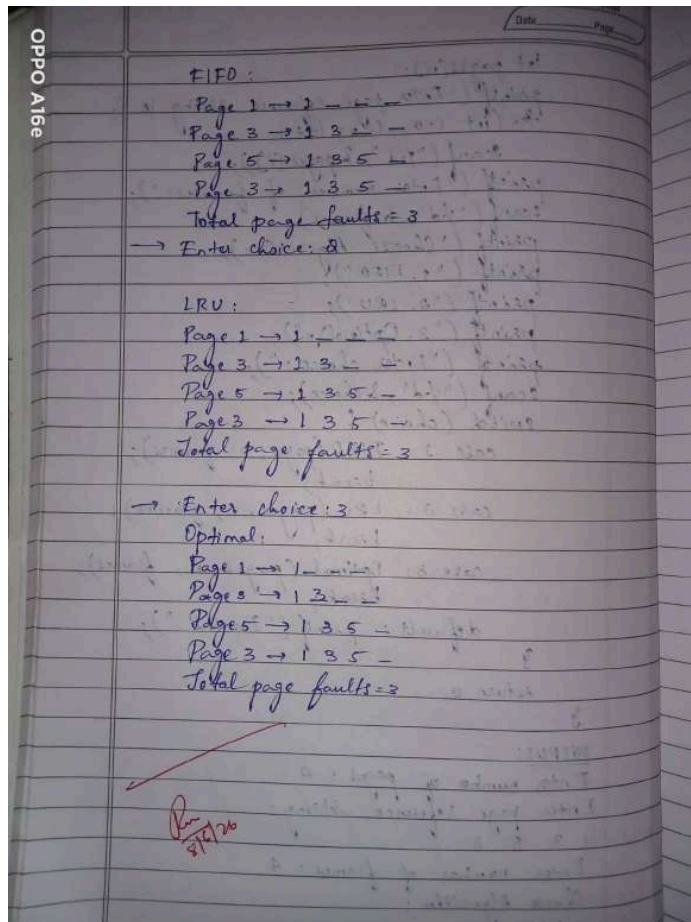
Default: C:\Users\anany\Desktop\OS\optimal.exe
ctrl+alt+t

C:\Users\anany\Desktop\OS>
Enter number of pages: 12
Enter page reference string:
1 2 3 4 1 2 5 1 2 3 4 5
Enter number of frames: 3

Page    Frames
1       1 - -
2       1 2 -
3       1 2 3
4       1 2 4
1       1 2 4
2       1 2 4
5       1 2 5
1       1 2 5
2       1 2 5
3       3 2 5
4       4 2 5
5       4 2 5
Total Page Faults = 7

```

Observation:



Github repository link

<https://github.com/ananya-ranjith/OS-LAB-PROGRAMS.git>